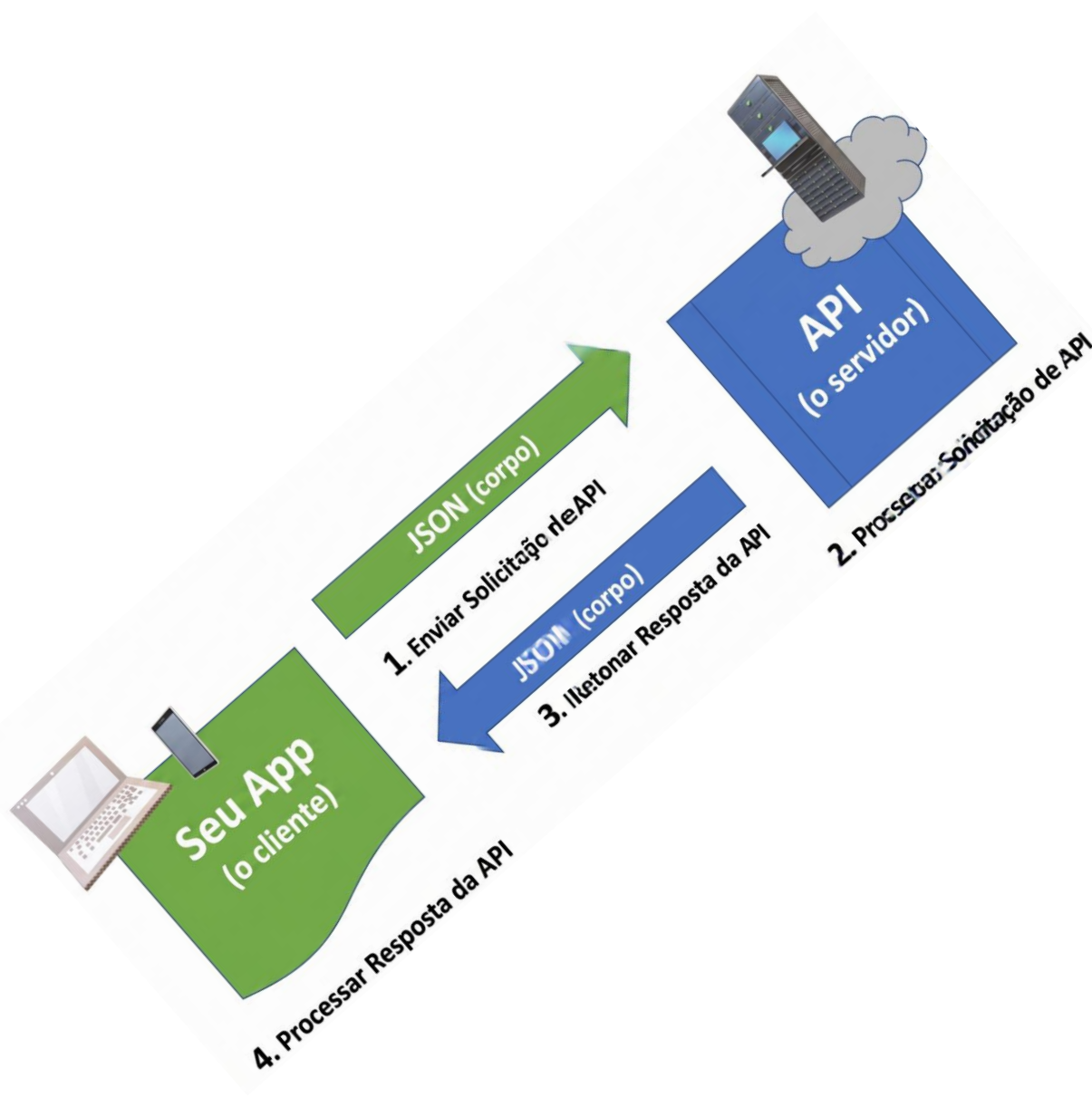


Interação com APIs

03

2026

SENAI - CRTI



Formatos de Dados e Contratos de Serviço

Como sistemas padronizam a troca de informações — JSON, XML, Swagger e Postman

Situação de Aprendizagem

O Desafio da Equipe Front-End

Uma empresa contratou uma equipe front-end para desenvolver uma aplicação de consulta de produtos. O backend já está pronto e documentado. Antes de escrever uma linha de código de tela, a equipe precisa responder:

Quais endpoints existem?

Quais rotas a API disponibiliza para consulta e manipulação de dados?

Quais dados retornam?

Qual é o formato e a estrutura exata de cada resposta do servidor?

Como enviar payloads?

Quais campos são obrigatórios? Quais tipos de dados são aceitos?

Quais formatos são aceitos?

O servidor aceita JSON? XML? Como indicar isso nos headers?

 **Pergunta-problema:** Como o front-end sabe exatamente quais dados enviar e quais dados esperar de uma API?

Contextualização Profissional

Integração Exige Padronização

No mercado de trabalho, sistemas diferentes precisam trocar dados o tempo todo. Veja como isso acontece na prática:



Marketplace

Recebe catálogo de produtos de múltiplos fornecedores em formato padronizado.



Banco Digital

Recebe dados cadastrais de clientes via API para abertura de conta.



App Escolar

Consome notas e frequência dos alunos em tempo real via endpoints REST.



ERP de Estoque

Integra movimentações de entrada e saída com o sistema financeiro corporativo.



API de Frete

Recebe endereço de entrega e devolve cálculo de prazo e custo automaticamente.

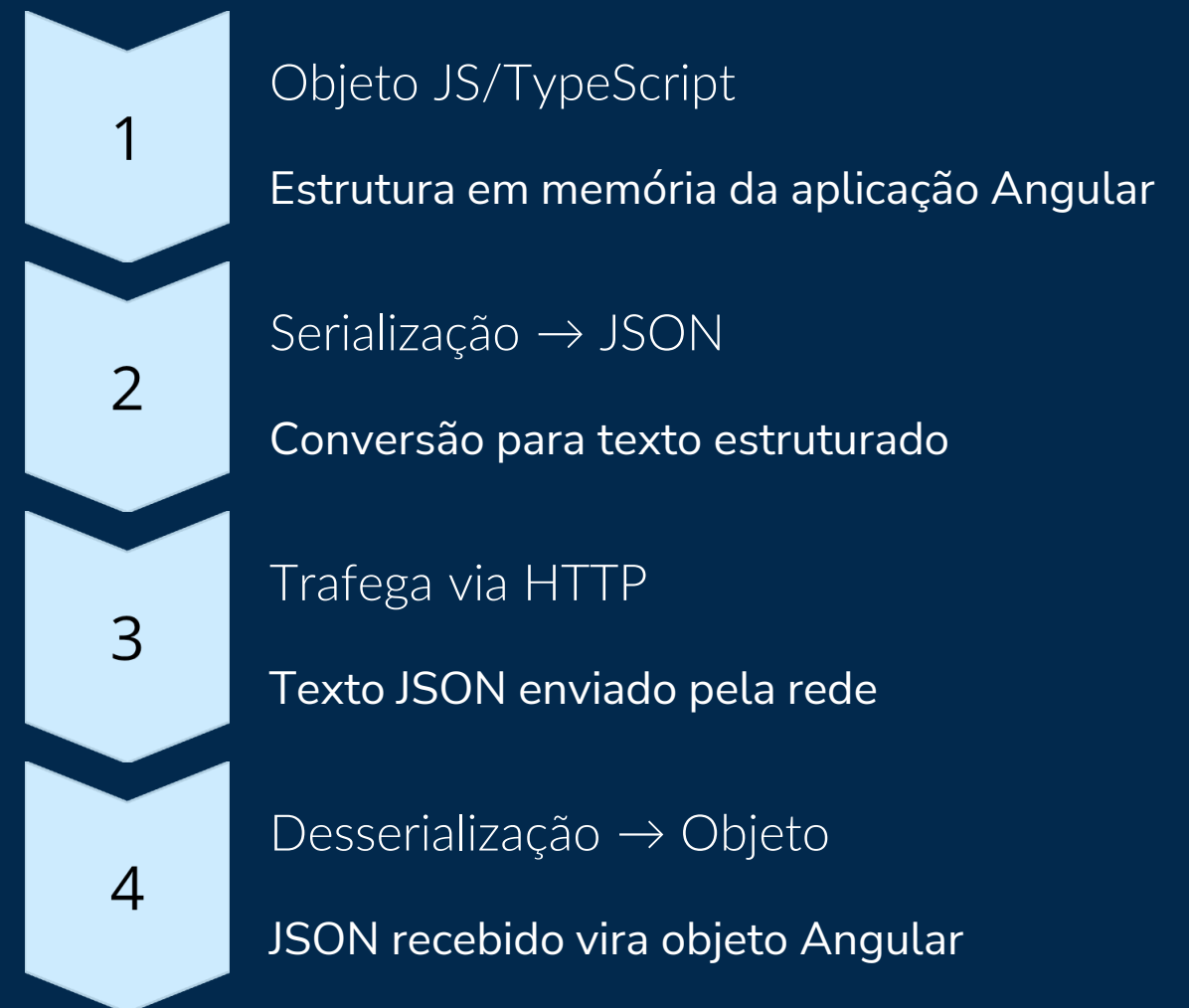
O que é Serialização?

Serialização é o processo de converter objetos ou estruturas de dados em um formato transmissível — texto, bytes ou outro formato padronizado que possa trafegar por uma rede.

Desserialização é o processo inverso: converter a resposta recebida (normalmente texto JSON) em um objeto manipulável dentro da aplicação Angular.

- ❏ Toda comunicação com APIs envolve esses dois processos. Entendê-los é essencial para depurar erros de integração.

Fluxo Conceitual



JSON – JavaScript Object Notation

JSON é o formato padrão dominante em APIs REST modernas. É **leve, legível por humanos, estruturado e suportado nativamente pelo JavaScript**. Praticamente toda API pública utiliza JSON como formato principal de troca de dados.



Leve



Legível



Estruturado



Padrão REST

Exemplo de objeto JSON:

```
{ "id": 1, "nome": "Notebook", "preco": 4500, "ativo": true }
```

Elemento	Tipo	Exemplo
id	number	1
nome	string	"Notebook"
preco	number	4500
ativo	boolean	true

JSON Array — Lista de Recursos

Quando uma API retorna **múltiplos recursos** (como uma lista de produtos, usuários ou pedidos), ela envolve os objetos dentro de um **array JSON** — delimitado por colchetes []. Cada item é um objeto separado por vírgula.

Usos comuns de JSON Array:

- Listagem de produtos
GET /produtos → retorna array com todos os itens
- Lista de usuários
GET /usuarios → retorna todos os cadastros
- Histórico de pedidos
GET /pedidos → retorna os pedidos do cliente
- Tarefas de um projeto
GET /tarefas → retorna todas as tarefas abertas

Exemplo de JSON Array:

```
[  
  {  
    "id": 1,  
    "nome": "Notebook"  
  },  
  {  
    "id": 2,  
    "nome": "Mouse"  
  }  
]
```

 O Angular processa arrays JSON como `any[]` ou como interfaces tipadas (`Produto[]`), permitindo iteração com `*ngFor` nos templates.

XML — Extensible Markup Language

O XML usa **tags hierárquicas** para estruturar dados, de forma similar ao HTML. É mais verboso que o JSON, mas ainda amplamente utilizado em **sistemas corporativos legados, integrações enterprise e protocolos como SOAP.**

Tags abertas e fechadas

Cada campo é envolvido por uma tag de abertura e fechamento

Hierarquia clara

Elementos filhos ficam dentro dos elementos pai

Verbosidade

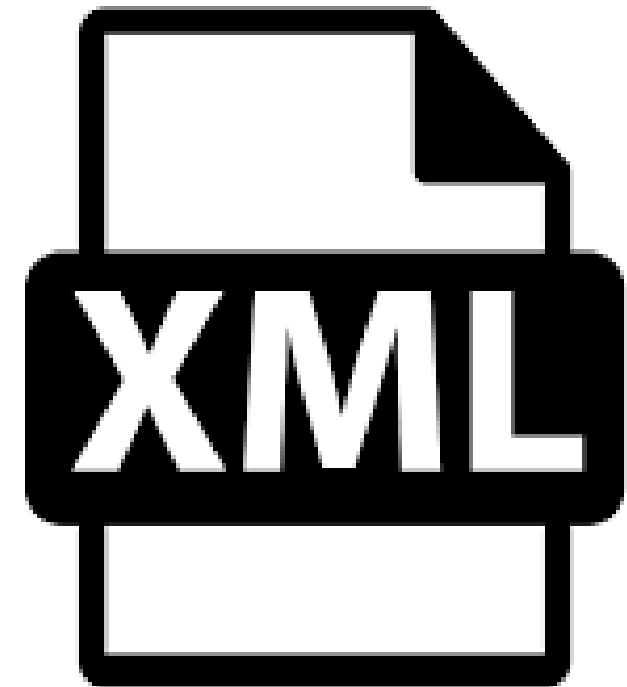
Mais caracteres que JSON para a mesma informação

Exemplo de objeto XML:

```
<produto>  
  <id>1</id>  
  <nome>Notebook</nome>  
  <preco>4500</preco>  
</produto>
```



XML ainda existe no mercado! Bancos, sistemas de governo, ERPs e integrações SOAP exigem XML. Todo desenvolvedor precisa saber reconhecê-lo e manipulá-lo.



JSON × XML: Qual Usar?

Ambos os formatos têm seu espaço no mercado. A escolha depende do contexto da integração, do sistema legado envolvido e dos requisitos do contrato de API.

Critério	JSON	XML
Legibilidade	✓ Alta, sintaxe simples	Moderada, mais verboso
Tamanho	✓ Menor (mais leve)	Maior (mais caracteres)
Velocidade	✓ Parse mais rápido	Parse mais lento
Uso em APIs REST	✓ Padrão dominante	Menos comum
Validação de schema	JSON Schema	✓ XSD/DTD robusto
Sistemas legados	Moderno	✓ Amplamente suportado
Protocolo SOAP	Não aplicável	✓ Obrigatório
Integração enterprise	Crescente	✓ Consolidado

👉 Para novas APIs REST com Angular, **JSON é sempre a escolha padrão**. Conheça XML para manutenção de sistemas legados e integrações corporativas existentes.

O que é um Contrato de API?

Definição

Um **contrato de API** é o documento formal que descreve como consumir um serviço. É o acordo entre o time de front-end e o time de back-end — define o que será enviado, o que será recebido e como os erros serão tratados.

Contrato = acordo entre front-end e back-end.
Sem contrato claro, a integração falha.

O contrato especifica:

1

Endpoint e Método

URL da rota + verbo HTTP (GET, POST, PUT, DELETE)

2

Parâmetros e Headers

Query params, path params, Content-Type, Authorization

3

Body e Schema

Estrutura do payload enviado e formato esperado

4

Retorno e Status Codes

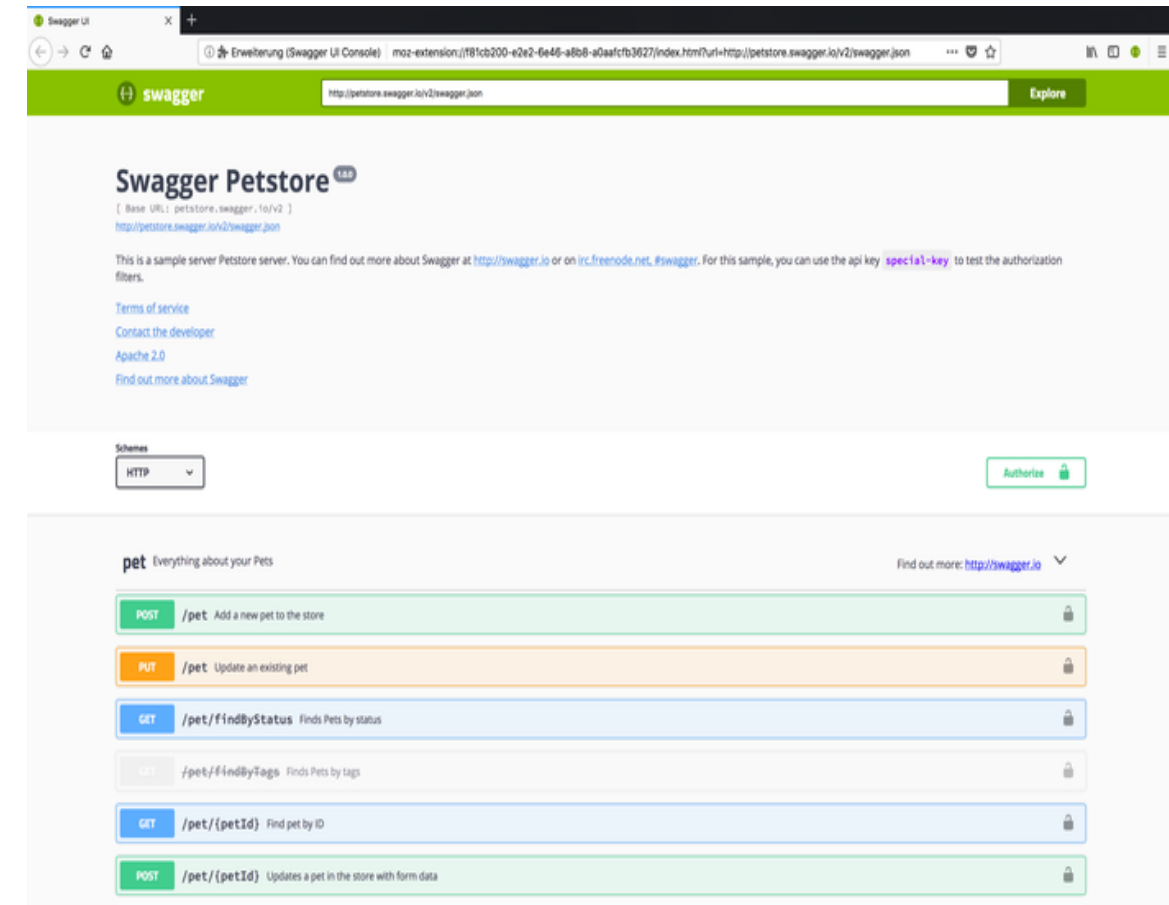
Estrutura da resposta + 200, 201, 400, 401, 404, 500

Swagger / OpenAPI

O **Swagger** (baseado na especificação **OpenAPI**) é a ferramenta de documentação de APIs mais utilizada no mercado. Ele transforma o contrato de API em uma interface interativa — onde o front-end consegue visualizar, entender e até **testar os endpoints diretamente no navegador**, antes de escrever qualquer linha de código Angular.

O Swagger documenta:

- Endpoints disponíveis
Todas as rotas com seus verbos HTTP
- Request e Response
Estrutura completa de entrada e saída
- Schemas de dados
Tipos, campos obrigatórios e exemplos
- Autenticação
Bearer token, API key, OAuth2



<https://petstore.swagger.io/>



Fluxo profissional: O back-end entrega a URL do Swagger → o front-end consulta a documentação → interpreta o contrato → implementa o serviço Angular.

Ferramenta do Mercado

Postman – O Laboratório de APIs

O **Postman** é a ferramenta padrão do mercado para testar, inspecionar e validar APIs durante o desenvolvimento. Ele permite que o front-end verifique se o contrato está funcionando corretamente antes de implementar o código Angular.

Testar Endpoints

Envie requisições GET, POST, PUT e DELETE para qualquer API sem escrever código.

Enviar Headers

Configure Content-Type, Authorization e outros headers necessários para autenticação.

Enviar Body

Monte payloads JSON ou XML e envie para endpoints que esperam dados no corpo da requisição.

Visualizar Retorno

Inspecione a resposta completa: status code, headers e body formatado.

Validar Contrato

Compare o retorno real com o que a documentação Swagger especifica.

Interpretando as Respostas da API

Todo contrato de API especifica os **status codes HTTP** esperados. O front-end precisa tratar cada situação corretamente no Angular para dar feedback adequado ao usuário.

2xx — Sucesso

200 OK: requisição atendida com sucesso.

201 Created: recurso criado (POST bem-sucedido).

204 No Content: sucesso sem retorno no body.

4xx — Erro do Cliente

400 Bad Request: payload inválido ou mal formatado.

401 Unauthorized: sem autenticação.

404 Not Found: recurso não encontrado.

5xx — Erro do Servidor

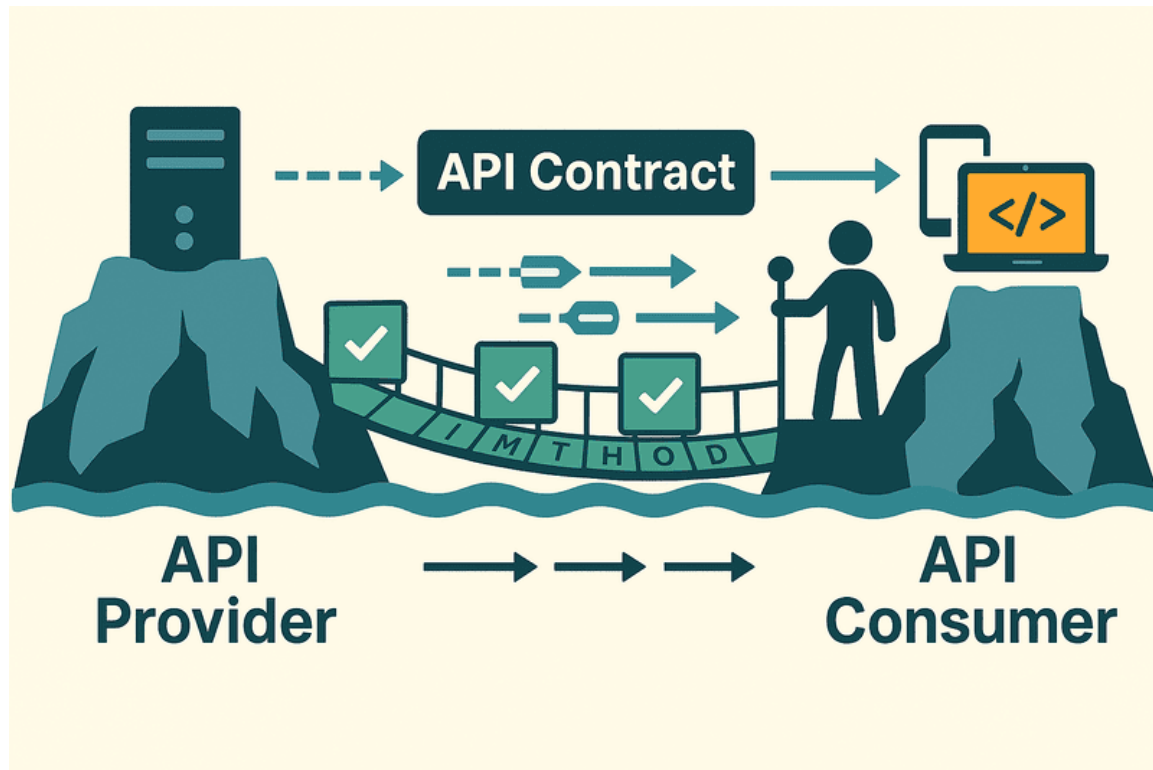
500 Internal Server Error: falha no back-end.

503 Service Unavailable: servidor indisponível.

⚠ O Angular trata esses erros no `catchError` do RxJS. Saber o código do erro ajuda a implementar mensagens adequadas na interface.

O que Acontece Quando o Contrato Falha?

Contratos de API mal definidos ou alterados sem comunicação são uma das principais causas de falhas em projetos de integração. Vamos debater cenários reais:



1

Campo renomeado

Se o back-end mudar "nome" para "descricao" sem avisar, o front-end exibe campos vazios — sem erro aparente, mas com dado incorreto.

2

Payload incorreto

Se o front-end enviar um campo errado ou ausente, a API retorna 400 Bad Request. Sem o contrato, o desenvolvedor não sabe o que está faltando.

3

Quem define o contrato?

Normalmente o back-end propõe, mas front-end e back-end devem validar juntos antes do desenvolvimento começar.

4

Contrato mal documentado

Gera retrabalho, reuniões desnecessárias, bugs em produção e atrasos na Entrega do projeto.

Analizando uma API Real com Postman

Etapa 1 — Ler o Contrato

Acesse a documentação da BrasilAPI ou outra API pública fornecida pelo instrutor. Identifique:

01

Rota e verbo HTTP

URL completa + GET, POST, etc.

02

Parâmetros exigidos

Path params, query params ou body

03

Formato de resposta

JSON ou XML? Quais campos?

04

Campos obrigatórios

O que é required no schema?

05

Status codes possíveis

200, 400, 404, 500?

Etapa 2 — Testar no Postman

Com o contrato em mãos, abra o Postman e:

- 1 Configure a requisição
Selecione o verbo HTTP e cole a URL do endpoint.
- 2 Adicione os headers necessários
Content-Type: application/json e outros.
- 3 Envie e analise a resposta
Compare o retorno real com o schema do contrato.
- 4 Registre suas descobertas
Anote campos, tipos e comportamentos observados.

Tipos de Dados no Schema JSON

Ao interpretar um contrato de API, o schema define exatamente o **tipo, formato e obrigatoriedade** de cada campo. Saber ler isso é fundamental para implementar a interface TypeScript correta no Angular.

Tipo JSON	TypeScript	Exemplo no JSON	Uso comum
string	string	"Notebook"	Nomes, textos, datas ISO
number	number	4500 ou 3.14	Preços, quantidades, IDs
boolean	boolean	true / false	Flags de status, ativo/inativo
object	interface / {}	{ "id": 1, "nome": "..."}	Recursos completos
array	any[] / T[]	[{...}, {...}]	Listas de recursos
null	null / undefined	null	Campos ausentes ou opcionais

 No Angular, criamos **interfaces TypeScript** que espelham o schema do contrato. Isso garante tipagem estática e evita erros em tempo de desenvolvimento.

Boas Práticas

Trabalhando com APIs de Forma Profissional

Leia o contrato
antes de codar

Sempre consulte a
documentação
Swagger antes de
implementar qualquer
serviço Angular. Isso
evita retrabalho.

Valide com
Postman primeiro

Teste o endpoint
manualmente e
confirme que a
resposta corresponde
ao que o contrato
especifica.

Crie interfaces
TypeScript

Modele os dados da
API como interfaces
tipadas no Angular
para aproveitar o
sistema de tipos e o
IntelliSense.

Trate todos os
status codes

Implemente
`catchError` no
serviço Angular para
tratar erros 4xx e 5xx
com mensagens
adequadas ao
usuário.

Revisão da Aula

Conectando os Conceitos



A jornada do front-end começa pelo contrato: entender o formato dos dados, como eles são serializados, o que a documentação especifica e como validar tudo isso com Postman antes de escrever o código Angular.

Evidência de Competência

O que Você é Capaz de Fazer Agora

Ao concluir esta aula, o aluno demonstra as seguintes competências profissionais, alinhadas ao perfil do desenvolvedor de sistemas do mercado de trabalho:

✓ Diferenciar JSON e XML

Reconhecer cada formato, suas características, vantagens e cenários de uso no mercado.

✓ Interpretar Payloads

Ler e compreender a estrutura de objetos JSON e arrays, identificando tipos e campos.

✓ Compreender Serialização

Explicar o processo de conversão de objetos para JSON e de JSON para objetos no fluxo HTTP.

✓ Ler Contratos de API

Identificar endpoints, verbos, parâmetros, schemas e status codes em um contrato.

✓ Navegar no Swagger

Usar a interface Swagger/OpenAPI para consultar e testar endpoints documentados.

✓ Testar no Postman

Configurar e executar requisições HTTP, inspecionar respostas e validar o contrato.

